

**UNIVERSITY OF SCIENCE AND
TECHNOLOGY (USTC), CHITTAGONG**
Department of Computer Science & Engineering (CSE)

LAB REPORT ASSIGNMENT

Course Name: Data Structure

Batch: 45 (B)

Semester: 3rd

SUBMITTED BY

Name: Mohammed Abtahi Rasul Sikder

Roll: 25070155

Batch: 45 (B)

Semester: 3rd

SUBMITTED TO

Name: Umme Aiman Jannat

Designation: Lecturer

Department: Computer Science &
Engineering

Submission Date: 15-09-2026

Task 1: Generate all permutations of a given set of elements

```
#include <stdio.h>
#include <string.h>

void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}

void permute(char *a, int l, int r) {
    if (l == r)
        printf("%s\n", a);
    else {
        for (int i = l; i <= r; i++) {
            swap((a + l), (a + i));
            permute(a, l + 1, r);
            swap((a + l), (a + i)); // backtrack
        }
    }
}

int main() {
    char str[100];
    printf("Enter string: ");
    scanf("%s", str);
    int n = strlen(str);
    printf("All Permutations:\n");
    permute(str, 0, n - 1);
    return 0;
}
```

Task 2: Solve a quadratic equation and display its roots

```
#include <stdio.h>
#include <math.h>

int main() {
    double a, b, c, discriminant, root1, root2, realPart, imagPart;
    printf("Enter coefficients a, b and c: ");
    scanf("%lf %lf %lf", &a, &b, &c);

    discriminant = b * b - 4 * a * c;

    if (discriminant > 0) {
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf("Root 1 = %.2lf and Root 2 = %.2lf\n", root1, root2);
    } else if (discriminant == 0) {
        root1 = root2 = -b / (2 * a);
        printf("Root 1 = Root 2 = %.2lf\n", root1);
    } else {
        realPart = -b / (2 * a);
        imagPart = sqrt(-discriminant) / (2 * a);
        printf("Root 1 = %.2lf + %.2lfi and Root 2 = %.2lf - %.2lfi\n", realPart, imagPart,
realPart, imagPart);
    }
    return 0;
}
```

Task 3: Find prime numbers using the Sieve of Eratosthenes method

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

void sieveOfEratosthenes(int n) {
    bool prime[n + 1];
    memset(prime, true, sizeof(prime));

    for (int p = 2; p * p <= n; p++) {
        if (prime[p] == true) {
            for (int i = p * p; i <= n; i += p)
                prime[i] = false;
        }
    }

    printf("Prime numbers up to %d: ", n);
    for (int p = 2; p <= n; p++) {
        if (prime[p])
            printf("%d ", p);
    }
    printf("\n");
}

int main() {
    int n;
    printf("Enter limit (n): ");
    scanf("%d", &n);
    sieveOfEratosthenes(n);
    return 0;
}
```

Task 4: Implement String-Handling Functions

```
#include <stdio.h>

int LENGTH(char str[]) {
    int i = 0;
    while (str[i] != '\0') i++;
    return i;
}

char GETCHAR(char str[], int n) {
    return str[n];
}

void PUTCHAR_FUNC(char str[], int n, char c) {
    str[n] = c;
}

int POS(char str1[], char str2[]) {
    int len1 = LENGTH(str1), len2 = LENGTH(str2);
    for (int i = 0; i <= len1 - len2; i++) {
        int j;
        for (j = 0; j < len2; j++) {
            if (str1[i + j] != str2[j]) break;
        }
        if (j == len2) return i;
    }
    return -1;
}

void CONCAT(char str1[], char str2[], char result[]) {
    int i = 0, j = 0;
    while (str1[i] != '\0') {
        result[i] = str1[i];
        i++;
    }
    while (str2[j] != '\0') {
        result[i + j] = str2[j];
        j++;
    }
    result[i + j] = '\0';
}

void SUBSTRING(char str[], int i, int m, char result[]) {
    int k;
    for (k = 0; k < m && str[i + k] != '\0'; k++) {
        result[k] = str[i + k];
    }
    result[k] = '\0';
}

void DELETE(char str[], int i, int m) {
    int len = LENGTH(str);
    for (int k = i; k + m <= len; k++) {
        str[k] = str[k + m];
    }
}

void INSERT(char str1[], int i, char str2[]) {
    char temp[200];
    int len1 = LENGTH(str1), len2 = LENGTH(str2);
```

```

int k;
for (k = 0; k < i; k++) temp[k] = str1[k];
for (int j = 0; j < len2; j++) temp[k++] = str2[j];
for (int j = i; j < len1; j++) temp[k++] = str1[j];
temp[k] = '\0';

int z = 0;
while(temp[z] != '\0') {
    str1[z] = temp[z];
    z++;
}
str1[z] = '\0';
}

int COMPARE(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] != str2[i]) return str1[i] - str2[i];
        i++;
    }
    return str1[i] - str2[i];
}

int main() {
    char s1[100] = "Hello World";
    char s2[100] = "World";
    char res[200];

    printf("Length of s1: %d\n", LENGTH(s1));
    printf("Char at index 1: %c\n", GETCHAR(s1, 1));

    PUTCHAR_FUNC(s1, 0, 'h');
    printf("After PUTCHAR: %s\n", s1);

    printf("Position of s2 in s1: %d\n", POS(s1, s2));

    SUBSTRING(s1, 6, 5, res);
    printf("Substring: %s\n", res);

    DELETE(s1, 0, 6);
    printf("After Delete: %s\n", s1);

    INSERT(s1, 0, "hello ");
    printf("After Insert: %s\n", s1);

    printf("Compare result: %d\n", COMPARE("abc", "abc"));
    return 0;
}

```

Task 5: Find a substring within a string and display its starting position

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], sub[100];
    printf("Enter main string: ");
    gets(str);
    printf("Enter substring: ");
    gets(sub);

    char *pos = strstr(str, sub);

    if (pos != NULL) {
        printf("Substring found at starting position/index: %ld\n", pos - str);
    } else {
        printf("Substring not found.\n");
    }
    return 0;
}
```

Task 6: Find the index of a specific pattern within a string

```
#include <stdio.h>
#include <string.h>

int findPatternIndex(char text[], char pattern[]) {
    int n = strlen(text);
    int m = strlen(pattern);

    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m)
            return i;
    }
    return -1;
}

int main() {
    char text[100], pattern[100];
    printf("Enter Text: ");
    scanf("%s", text);
    printf("Enter Pattern: ");
    scanf("%s", pattern);

    int index = findPatternIndex(text, pattern);
    if (index != -1)
        printf("Pattern found at index: %d\n", index);
    else
        printf("Pattern not found.\n");

    return 0;
}
```

Task 7: Concatenate two strings

```
#include <stdio.h>

int main() {
    char str1[100], str2[100], result[200];
    int i = 0, j = 0;

    printf("Enter first string: ");
    scanf("%s", str1);
    printf("Enter second string: ");
    scanf("%s", str2);

    while (str1[i] != '\0') {
        result[i] = str1[i];
        i++;
    }

    while (str2[j] != '\0') {
        result[i] = str2[j];
        i++;
        j++;
    }
    result[i] = '\0';

    printf("Concatenated String: %s\n", result);
    return 0;
}
```

Task 8: Insert a given text into a string at a specified position

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], text[50], result[150];
    int pos, i, j = 0;

    printf("Enter main string: ");
    scanf("%s", str);
    printf("Enter text to insert: ");
    scanf("%s", text);
    printf("Enter position: ");
    scanf("%d", &pos);

    for (i = 0; i < pos; i++) {
        result[j++] = str[i];
    }
    for (int k = 0; k < strlen(text); k++) {
        result[j++] = text[k];
    }
    for (i = pos; i < strlen(str); i++) {
        result[j++] = str[i];
    }
    result[j] = '\0';

    printf("Result string: %s\n", result);
    return 0;
}
```

Task 9: Delete a given text from a string at a specified position

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int pos, len;

    printf("Enter string: ");
    scanf("%s", str);
    printf("Enter starting position to delete: ");
    scanf("%d", &pos);
    printf("Enter length of text to delete: ");
    scanf("%d", &len);

    int total_len = strlen(str);
    for (int i = pos; i + len <= total_len; i++) {
        str[i] = str[i + len];
    }

    printf("Modified string: %s\n", str);
    return 0;
}
```

Task 10: Sort an array using the Bubble Sort algorithm

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    printf("Sorted array (Bubble Sort): \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Task 11: Sort an array using the Insertion Sort algorithm

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);

    printf("Sorted array (Insertion Sort): \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Task 12: Apply Linear Search to find a given element in an array

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 80, 30, 60, 50, 110, 100, 130, 170};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 110;

    int result = linearSearch(arr, n, target);
    if (result == -1)
        printf("Element is not present in array\n");
    else
        printf("Element is present at index %d\n", result);
    return 0;
}
```

Task 13: Apply Binary Search to find a given element in an array

```
#include <stdio.h>

int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        int m = l + (r - l) / 2;

        if (arr[m] == x)
            return m;
        if (arr[m] < x)
            l = m + 1;
        else
            r = m - 1;
    }
    return -1;
}

int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;

    int result = binarySearch(arr, 0, n - 1, x);
    if (result == -1)
        printf("Element is not present in array\n");
    else
        printf("Element is present at index %d\n", result);
    return 0;
}
```

Task 14: Apply Binary Search to find the first occurrence, last occurrence, and total count of a given element in an array

```
#include <stdio.h>

int firstOccurrence(int arr[], int n, int x) {
    int low = 0, high = n - 1, res = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            res = mid;
            high = mid - 1; // search left
        } else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return res;
}

int lastOccurrence(int arr[], int n, int x) {
    int low = 0, high = n - 1, res = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            res = mid;
            low = mid + 1; // search right
        } else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return res;
}

int main() {
    int arr[] = {1, 2, 2, 2, 2, 3, 4, 7, 8, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 2;

    int first = firstOccurrence(arr, n, x);
    int last = lastOccurrence(arr, n, x);

    if (first == -1) {
        printf("Element not found\n");
    } else {
        printf("First Occurrence = %d\n", first);
        printf("Last Occurrence = %d\n", last);
        printf("Total Count = %d\n", (last - first + 1));
    }
    return 0;
}
```